

COMPLESSITÀ ASINTOTICA

O-Grande: $f(n) \in O(g(n)) \Leftrightarrow \exists c > 0, \exists n_0$, tale che $\forall n > n_0, f(n) \leq c \cdot g(n)$

una funzione $f(n)$ è $O(g(n))$ (fa parte di tale insieme) se e solo se esistono due costanti $c > 0$ e n_0 tali che per qualunque $n > n_0$, $f(n) \leq c \cdot g(n)$.

Detto in altre parole: a parte un fattore moltiplicativo c e un numero finito di casi (quando $n > n_0$, e quindi asintoticamente), $f(n)$ cresce al massimo come $g(n)$.

Quindi: questa definizione ci permette di utilizzare $g(n)$ come **limite superiore** di $f(n)$ in senso **asintotico** ed ignorando un fattore moltiplicativo

Ω : $f(n) \in \Omega(g(n)) \Leftrightarrow \exists c > 0, \exists n_0$, tale che $\forall n > n_0, f(n) \geq c \cdot g(n)$

una funzione $f(n)$ è $\Omega(g(n))$ (fa parte di tale insieme) se e solo se esistono due costanti $c > 0$ e n_0 tali che per qualunque $n > n_0$, $f(n) \geq c \cdot g(n)$.

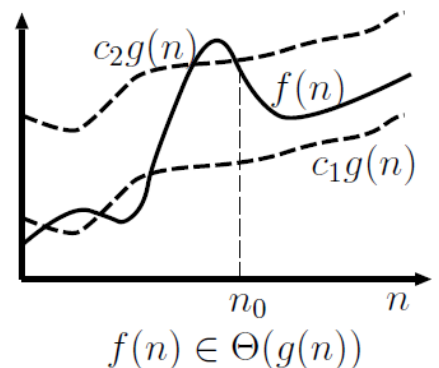
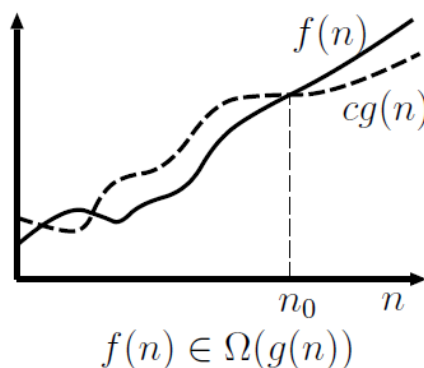
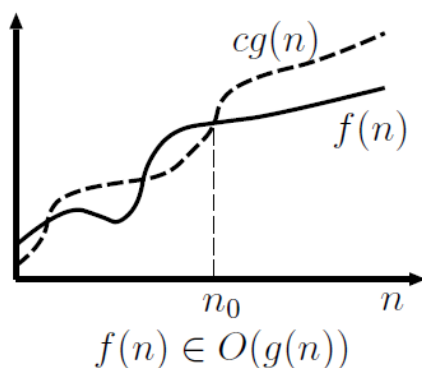
Detto in altre parole: a parte un fattore moltiplicativo c e un numero finito di casi (quando $n > n_0$, e quindi asintoticamente), $f(n)$ cresce almeno come $g(n)$.

Quindi: questa definizione ci permette di utilizzare $g(n)$ come **limite inferiore** di $f(n)$ in senso **asintotico** ed ignorando un fattore moltiplicativo

Θ : $f(n) \in \Theta(g(n)) \Leftrightarrow \exists c_1 > 0, \exists c_2 > 0, \exists n_0$, tale che $\forall n > n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Detto in altre parole: a parte fattori moltiplicativi e un numero finito di casi (quando $n > n_0$, e quindi asintoticamente), $f(n)$ cresce come $g(n)$.

Quindi: questa definizione ci permette di utilizzare $g(n)$ come **limite inferiore e superiore** di $f(n)$ in senso **asintotico** ed ignorando un fattore moltiplicativo.



Queste definizioni danno un'idea precisa di cosa sono **O** , **Ω** e **Θ** ma possono essere difficili da utilizzare. Pertanto, viene utilizzato il seguente teorema:

Teorema: sia $a = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$; se esiste, allora:

- $0 \leq a < \infty \Leftrightarrow f(n) \in O(g(n))$
- $0 < a \leq \infty \Leftrightarrow f(n) \in \Omega(g(n))$
- $0 < a < \infty \Leftrightarrow f(n) \in \Theta(g(n))$

Esercizio 1:

Si stabilisca, dimostrandolo formalmente, se le tre funzioni 2^{n-100} , $2^{\frac{1}{2}n}$, $\left(\frac{3}{2}\right)^n$ siano $\Omega(2^n)$.

- $2^{n-100} = 2^n / 2^{100}$ dove $1/2^{100}$ è solo un fattore moltiplicativo e dunque $2^{n-100} \in \Omega(2^n)$.
La stessa cosa si può dimostrare utilizzando il teorema visto pocanzi:

$$\lim_{n \rightarrow +\infty} \frac{2^{n-100}}{2^n} = \lim_{n \rightarrow +\infty} \frac{1}{2^{100}} > 0 \text{ e dunque } 2^{n-100} \in \Omega(2^n)$$

- $2^{\frac{1}{2}n}$: $\lim_{n \rightarrow +\infty} \frac{2^{\frac{1}{2}n}}{2^n} = \lim_{n \rightarrow +\infty} \frac{(\sqrt{2})^n}{2^n} = \lim_{n \rightarrow +\infty} \left(\frac{\sqrt{2}}{2}\right)^n = 0$ e dunque $2^{\frac{1}{2}n} \notin \Omega(2^n)$
perchè $\sqrt{2} < 2$

- $\left(\frac{3}{2}\right)^n$: $\lim_{n \rightarrow +\infty} \frac{\left(\frac{3}{2}\right)^n}{2^n} = \lim_{n \rightarrow +\infty} \left(\frac{\frac{3}{2}}{2}\right)^n = 0$ e dunque $\left(\frac{3}{2}\right)^n \notin \Omega(2^n)$
perchè $3/2 < 2$

Esercizio 2:

Si dimostri la seguente implicazione: $f_1(n) \in O(g_1(n)) \wedge f_2(n) \in O(g_2(n)) \Rightarrow f_1(n) + f_2(n) \in O(g_1(n) + g_2(n))$

- $f_1(n) \in O(g_1(n))$ implica che esistono due costanti $c_1 > 0$ e n_1 tale che $\forall n > n_1, f_1(n) \leq c_1 g_1(n)$
- $f_2(n) \in O(g_2(n))$ implica che esistono due costanti $c_2 > 0$ e n_2 tale che $\forall n > n_2, f_2(n) \leq c_2 g_2(n)$
- A questo punto, scegliendo due costanti $n_0 = \max(n_1, n_2)$ e $c = \max(c_1, c_2)$ possiamo dire che $\forall n > n_0, f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c g_1(n) + c g_2(n) = c(g_1(n) + g_2(n))$ e quindi abbiamo stabilito che $f_1(n) + f_2(n) \in O(g_1(n) + g_2(n))$

Esercizio 3:

Dimostrare che la seguente implicazione è falsa: $f_1(n) \in O(g_1(n)) \vee f_2(n) \in O(g_2(n)) \Rightarrow f_1(n) + f_2(n) \in O(g_1(n) + g_2(n))$

Basta trovare come controesempio 4 funzioni per cui non valva tale implicazione

- Ad esempio con: $f_1(n) = n^2, f_2(n) = n, g_1(n) = n, g_2(n) = n$
l'implicazione non vale poiché $f_1(n) + f_2(n) = n^2 + n \notin O(2n) = O(g_1(n) + g_2(n))$

CORRETTEZZA TRAMITE INVARIANTE

La correttezza degli algoritmi iterativi può essere dimostrata utilizzando gli **Invarianti**.

Un Invariante è una *proposizione* che esprime una proprietà delle variabili dell'algoritmo che resta sempre vero in vari punti dell'algoritmo.

Nell'ambito di un ciclo, l'Invariante di Ciclo deve soddisfare 2 caratteristiche:

1. **inizializzazione**: la *proposizione* vale immediatamente prima di entrare nel ciclo
2. **mantenimento**: se la *proposizione* vale prima di eseguire il corpo del ciclo, allora vale anche dopo aver eseguito il corpo del ciclo

Inoltre, l'Invariante deve essere **utile**: cioè, deve essere significativo per quanto riguarda la correttezza dell'algoritmo.

In particolare, quello che si fa è stabilire i valori delle variabili nel momento in cui il ciclo termina; esaminare la *proposizione* e giungere alla conclusione se questo ci porta verso la dimostrazione della correttezza dell'algoritmo o meno

Esercizio:

Dimostrare la correttezza del seguente algoritmo per calcolare il valore di un polinomio rappresentato dai suoi coefficienti $a_0, a_1, \dots, a_n$ in un punto x .

HORNER($a_0, a_1, \dots, a_n, x$)

```

y ← 0
i ← n
while i ≥ 0 do
    y ← ai + x · y
    i ← i - 1
return y
    
```

Per dimostrare la correttezza di questo algoritmo utilizziamo un invariante.

Per scrivere una proposizione che sui valori che entrano in gioco (i e y) che valga prima, durante e dopo il ciclo, può essere utile simulare le esecuzioni del ciclo:

# esecuzioni del ciclo	i	y
0	n	0
1	$n-1$	a_n
2	$n-2$	$a_{n-1} + a_n x$
3	$n-3$	$a_{n-2} + a_{n-1}x + a_n x^2$
4	$n-4$	$a_{n-3} + a_{n-2}x + a_{n-1}x^2 + a_n x^3$
⋮	⋮	



Si evince che la relazione fra i e y in ogni riga della tabella precedente, cioè l'invariante del ciclo, è:

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k$$

Per dimostrare che questa uguaglianza sia un invariante dobbiamo dimostrare che l'*inizializzazione* ed il *mantenimento* ... tuttavia omettiamo questa parte! ...

... ciò che facciamo invece è dimostrarne l'*utilità*: correttezza dell'algoritmo!

- Detto in altre parole dobbiamo vedere quanto vale y quando il ciclo termina... e per fare questo dobbiamo capire quanto vale i all'uscita del ciclo: $i = -1$

▫ Dunque:

$$y = \sum_{k=0}^{n-(-1+1)} a_{k-1+1} x^k = \sum_{k=0}^n a_k x^k \text{ e quindi l'algoritmo restituisce la soluzione corretta!}$$

RELAZIONI DI RICORRENZA

Parliamo ora di come stabilire la complessità temporale di un algoritmo utilizzando le **Relazioni di Ricorrenza**.

Esercizio:

Si analizzi la complessità temporale in funzione di n del seguente algoritmo, ricavando la relazione di ricorrenza e calcolandone il Θ :

```
BUMP( $n$ )
  if  $n \leq 1$  then return 5
  else
     $m \leftarrow 0$ 
    for  $i \leftarrow 1$  to 8 do
       $m \leftarrow m + \text{BUMP}(\lfloor n/2 \rfloor)$ 
    for  $i \leftarrow 1$  to  $n$  do
      for  $j \leftarrow 1$  to  $n$  do
         $m \leftarrow m + 1$ 
    return  $m$ 
 $n^2$ .
```

- il corpo dei *for* annidati viene eseguito n^2 volte;
- BUMP($\lfloor n/2 \rfloor$) viene eseguito 8 volte in maniera ricorsiva.

Da ciò si ottiene la ricorrenza:

$$T(n) = \begin{cases} 8 \cdot T(n/2) + n^2 & \text{se } n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

svolgendo:

$T(n) = 8T(n/2) = 8(8 \cdot T(n/2^2) + (n/2)^2) + n^2 = 8^2 T(n/2^2) + 2n^2 + n^2 = \dots$
da cui, in generale, quando $k \leq \log_2 n$ (scritto semplicemente $\log n$
nel seguito): $T(n) = 8^k T(n/2^k) + (\sum_{i=0}^{k-1} 2^i) \cdot n^2 = 8^k T(n/2^k) + (2^k - 1) \cdot$

Per applicare il caso base (cioè, quando $n = 1$ e $T(n) = 1$), dobbiamo considerare la formula precedente con $k = \log n$. Otteniamo: $T(n) = 8^{\log n} + (2^{\log n} - 1) \cdot n^2 = 2n^3 - n^2$ e dunque $\Theta(n^3)$.

ANALISI DI UN ALGORITMO

Esercizio:

Si consideri il seguente algoritmo:

```
AUGUSTA(A[1..n])
  for  $i \leftarrow n - 1$  down to 1 do
     $j \leftarrow i$ 
    while  $j < n$  and  $A[j] < A[j + 1]$  do
      scambia  $A[j + 1]$  con  $A[j]$ 
       $j \leftarrow j + 1$ 
```

... e si risponda alle seguenti domande:

- 1) cosa fa AUGUSTA($A[1..n]$)?
- 2) si indichino l'invariante del ciclo esterno e del ciclo interno.
- 3) quali sono il caso peggiore e la sua complessità in termini di Θ ?
- 4) sapreste indicare un algoritmo che risolva lo stesso problema in tempo asintoticamente migliore?

L'algoritmo in questione:

- prende in input un vettore di n elementi;
- è composto da due cicli:
 - ciclo esterno *for*:
 - in cui i parte da $n - 1$ e viene decrementato fino all'ultimo valore 1
 - assegna alla variabile j il valore di i
 - ciclo interno *while*:
 - in cui la condizione di permanenza è che $j < n$ e $A[j] < A[j + 1]$
 - se ciò si verifica: si scambiano i valori di $A[j + 1]$ con $A[j]$ e j viene incrementato di 1

1) AUGUSTA ordina in senso **non crescente** il vettore $A[1..n]$ con una variante dell'INSERT - SORT

L'idea infatti è prendere l'elemento in posizione i ed inserirlo nella parte già ordinata in senso non crescente che si trova in fondo al vettore

2) invariante esterno: $A[i + 1..n]$ è ordinato in senso non crescente

invariante interno: $\forall h \in \{i..j - 1\}, A[h] \geq A[j]$

3) il caso peggiore si verifica quando A sia ordinato in senso crescente, e l'algoritmo è $\Theta(n^2)$

4) una soluzione migliore è un'opportuna variante di un algoritmo $\Theta(n \log n)$, come il MARGE - SORT

STRUTTURE DATI

Esercizio 1:

Si consideri una *tabella di hash ad indirizzamento aperto* $T[0, \dots, 19]$, con 20 elementi, che utilizza la seguente funzione di hashing $h(k, i) = (k \bmod 20 + 3i + 2i^2) \bmod 20$.

1) si riporti il suo contenuto dopo l'inserimento delle seguenti chiavi: 15, 40, 35, 20

2) si riporti l'algoritmo di ricerca di una chiave in una tabella hash che usa l'indirizzamento aperto

Domanda 1:

Prima di vedere la soluzione, riportiamo l'algoritmo di inserimento di una chiave in una tabella di hash ad *indirizzamento aperto* (NB: non richiesto dalla traccia):

```
HASH-INSERT( $k, T[0..m-1]$ )  
  for  $i \leftarrow 0$  to  $m-1$  do  
     $j \leftarrow h(k, i)$   
    if  $T[j] \neq \text{nil}$  then  
       $T[j] \leftarrow k$   
      return  
  error overflow
```

L'algoritmo prende in input una chiave che deve essere inserita nella tabella di hash.

L'inserimento avviene durante un ciclo... in cui, ad ogni ciclo si calcola la funzione di hash; se tale valore corrisponde ad una posizione non *nil*, allora la chiave viene inserita in tale posizione.

Quindi l'indice del ciclo rappresenta il numero di collisioni che

la chiave subisce durante questi tentativi di inserimento.

- Nel nostro caso $m = 20$
- all'inizio $T[j] = \text{nil}$ per ogni $j \in 0..19$
- Segue che:

Inserimento di 15: viene inserito in $h(15, 0) = 15$

Inserimento di 40: viene inserito in $h(40, 0) = 0$

Inserimento di 35: $h(35, 0) = 15$, ma $T[15] = 15$; allora

$h(35, 1) = (15 + 5) \bmod 20 = 0$, ma $T[0] = 40$; infine, viene inserito in:

$h(35, 2) = (15 + 6 + 8) \bmod 20 = 9$

Inserimento di 20: $h(20, 0) = 0$, ma $T[0] = 40$; allora, viene inserito in:

$h(20, 1) = (3 + 2) \bmod 20 = 5$

Domanda 2:

l'algoritmo di ricerca di una chiave in una tabella hash che usa l'indirizzamento aperto è:

```
HASH-SEARCH( $k, T[0..m-1]$ )  
  for  $i \leftarrow 0$  to  $m-1$  do  
     $j \leftarrow h(k, i)$   
    if  $T[j] = \text{nil}$  then  
      return nil  
    else  
      if  $T[j].\text{key} = k$  then  
        return  $T[j]$   
  return nil
```

L'algoritmo di ricerca ritorna l'elemento di T con chiave k se esiste, *nil* altrimenti.

Esercizio 2:

1) riportare le caratteristiche di un *heap minimo*

2) il seguente array rappresenta un heap minimo (1, 2, 3, 10, 8, 9, 7, 14, 15, 16, 13); effettuare l'estrazione del minimo riportando lo heap dopo ogni scambio di elementi

Domanda 1:

Uno heap minimo è un albero binario semi-completo sinistro (completo sino al penultimo livello mentre ciascun vertice dell'ultimo non ha lacune alla sua sinistra); inoltre la chiave di ogni vertice padre è $\leq$ di quella dei suoi eventuali figli.

Domanda 2:

Per rispondere è utile ricordarsi che nella rappresentazione in forma di array di uno heap minimo H , l'eventuale figlio sinistro di $H[i]$ è $H[2i]$ ed il suo eventuale figlio dx è $H[2i + 1]$.

Vediamo ora l'estrazione del minimo passo passo:

Passo 1: sostituire il primo elemento del vettore con l'ultimo e togliere l'ultimo elemento:

(1, 2, 3, 10, 8, 9, 7, 14, 15, 16, 13) -> (13, 2, 3, 10, 8, 9, 7, 14, 15, 16)

Passo 2: effettuo scambi per ristabilire lo heap:

Detto in altre parole, dobbiamo confrontare la chiave 13 con la chiave che si trova nei suoi figli sx/dx. Se il vincolo (padre $\leq$ figli) allora, nello scendere lungo l'albero, si farà lo scambio con la chiave più piccolo tra le chiavi dei suo figli:

(13, 2, 3, 10, 8, 9, 7, 14, 15, 16) ->

effetto scambio tra 13 e 2 (2, 13, 3, 10, 8, 9, 7, 14, 15, 16);

effetto scambio tra 13 e 8 (2, 13, 3, 10, 8, 9, 7, 14, 15, 16);

è finalmente uno heap minimo (2, 8, 3, 10, 13, 9, 7, 14, 15, 16).

GRAFI

Esercizio 1:

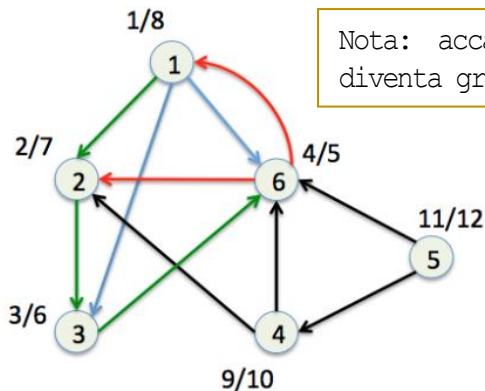
Si effettui la visita in profondità, a partire dal nodo 1, del seguente grafo di sei nodi dato tramite le liste di adiacenza.

Si disegni la foresta generata della visita, riportando per ogni nodo i tempi di inizio e fine visita (ossia i valori degli attributi d ed f) e per ogni arco il suo tipo secondo la classificazione degli archi durante una visita in profondità.

1:	2, 3, 6
2:	3
3:	6
4:	2, 6
5:	4, 6
6:	1, 2

La visita in profondità (DFS) può essere effettuata in due modi analoghi:

- tramite la lista delle adiacenze
- disegnando prima il grafo



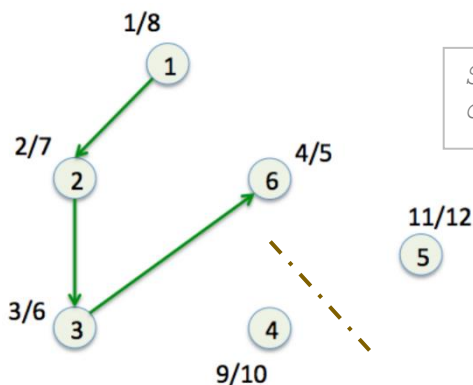
si parte dal nodo 1, poi si passa al nodo 2 (prima adiacente), poi al 3 e poi al 6, il quale ha due adiacenti entrambi nodi grigi (e non bianchi): dunque la visita per quanto riguarda il nodo 6 termina ($f = 5$).

Si torna al nodo 3 che non ha più nodi adiacenti: dunque la visita per quanto riguarda il nodo 3 termina ($f = 6$).

Si torna al 2: idem, ($f = 7$). Si torna al nodo 1: idem, ($f = 8$).

Si riparte dal nodo 4 ($d = 9$) che termina subito non avendo adiacenti bianchi. Si riparte dal 5, idem, ($f = 12$).

La foresta generata dalla visita è dunque la seguente:



Si osservi che la foresta generata contiene 3 alberi: il primo con 4 nodi e due alberi con un singolo nodo

Vediamo ora la classificazione degli archi:

- in verde gli archi che ci hanno fatto scoprire dei nodi: (1,2), (2,3) e (3,6)
- in rosso gli archi all'indietro che collegano un nodo al suo antenato: (6,1) e (6,2)
- in celeste gli archi in avanti che collegano un nodo al suo discendente: (1,3) e (1,6)
- in nero gli archi trasversali (o di attraversamento) che collegano o due alberi distinti o due rami distinti dello stesso albero: (4,2), (4,6), (5,4) e (5,6).

Esercizio 2:

A : B(3), C(6), D(7)

B : C(2), E(1)

C : D(1)

D :

E : D(4)

Si applichi l'algoritmo di Dijkstra al grafo riportato sotto con le liste di adiacenti a partire dal nodo A.

Riportare il contenuto della coda di priorità prima di ogni estrazione.

Quelli che seguono sono gli stati della coda (rappresentata come uno heap minimo in forma di array) prima dell'estrazione di ciascun vertice:

(A, 0), (B, ∞), (D, ∞), (C, ∞), (E, ∞)

(B, 3), (D, 7), (C, 6), (E, ∞)

(E, 4), (D, 7), (C, 5)

(C, 5), (D, 7)

(D, 6)

Il nodo di partenza è A: $d = 0$ mentre tutti gli altri d degli altri nodi sono infinito.

Si prosegue estraendo il nodo con attributo più piccolo ed eventualmente aggiornare gli attributi degli altri elementi (se ci sono cammini adiacenti più convenienti).

1) estraggo A: ha 3 adiacenti (B,C,D); tutti e tre i cammini sono cammini migliori (rispetto all'attuale infinito).

2) estraggo B: ha 2 adiacenti (C,E); tutti e due i cammini sono cammini migliori ($3+2$ e $3+1$).

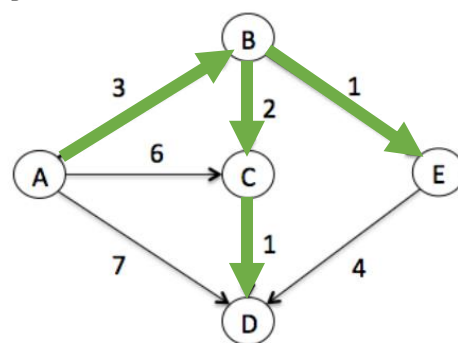
3) estraggo E: ha 1 adiacente (D); il cammino che abbiamo è già migliore rispetto a questo (8).

4) estraggo C: ha 1 adiacente (D); abbiamo trovato un cammino migliore ($5+1$).

5) estraggo D.

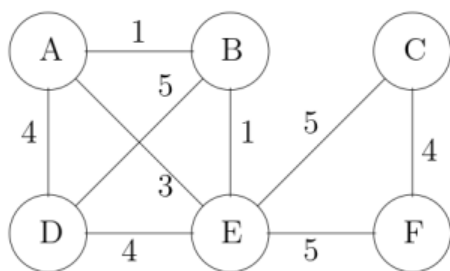
Graficamente:

gli archi colorati sono quelli che compongono i cammini minimi da A scelti dall'algoritmo di Dijkstra.



Esercizio 3:

Si disegnino tutti gli alberi minimi ricoprenti (MST) del grafo seguente



Per risolvere il quesito si può utilizzare analogamente uno tra algoritmo di Kruskal o Prim e ottenere la stessa soluzione. I possibili MST, il cui peso è 15, sono i seguenti:

